

Build a Website – Class 2

Xavier Esteve - Volunteer

Broward County SW Regional Library

What we covered previously

- **HTML Editing**
- **Some Styling with css**

What will be covered in this lesson

- Basics of Javascript
- Using Node.js to run a site in real-time

What is Node.js?

It can handle many connections at once without waiting for one to finish before starting another. Making it great for real-time apps and websites. Here are some examples of what you can build with Node.js:

- Web servers and websites
- REST APIs
- Real-time apps (like chat)
- Command-line tools
- Working with files and databases

How to start using Node.js

Download and Install Node.js

- Go to <https://nodejs.org>
- Download the LTS (Long Term Support) version
- Run the installer and follow the instructions

Verify Installation

- Open your terminal/cmd and type
 - `node --version`
 - `npm --version`

Download

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support
for EOL Node.js versions

Download Node.js®

Get Node.js® for using with

Info Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

Info Installation methods that involve community software are supported by the teams maintaining that software.

```
1 # On most Unix systems including macOS, you can install with a single command:
2 winget install Volta.Volta
3
4 # Download and install Node.js:
5 volta install node@22
6
7 # Verify the Node.js version:
8 node -v # Should print "v22.19.0".
9
10 # Verify npm version:
11 npm -v # Should print "10.9.3".
```

PowerShell

 Copy to clipboard

Volta is a cross-platform Node.js version manager. If you encounter any issues please visit Volta's website.

Or get a prebuilt Node.js® for running a architecture.

 Windows Installer (.msi)

 Standalone Binary (.zip)

Getting Started

- Create a file `myfirst.js` and add the following code

```
1  JS myfirst.js      x |
2  let http = require('http');
3  // "let" is used to define variables.
4  // "Require" will load a module, in this case, the 'http' modules is loaded
5
6  function greetUser(req, res) {
7      res.writeHead(200, {'Content-Type': 'text/html'});
8      res.end('Hello World!');
9  }
10
11 http.createServer(greetUser).listen(8080);
```

Getting Started Pt. 2

Initiate the Node.js file: ``node myfirst.js`` and open browser.



Reviewing Javascript

At a high level Javascript applications are made up of *statements* and *expressions*.

Statements

A statement is a unit of instruction made up of one or more lines of code that represent an action. For example, you can use the following statement to assign a value to a variable named myVariable:

```
let myVariable = 4;  
myVariable;  
> 4
```

Block statements

A block statement groups any number of statements and declarations inside a pair of braces ({}). It lets you combine statements in places where JavaScript expects only one.

You'll most frequently see block statements alongside [control flow](#) statements, such as **if**:

```
if ( x === 2 ) {  
    //some behavior;  
}
```

Reviewing Javascript

Expressions

An expression is a unit of code that results in a value, and can therefore be used wherever a value is expected. $2 + 2$ is an expression that results in the value 4:

$2 + 2$;

> 4

Data types and structures

Data types and structures

Primitives are the simplest types of data in JavaScript. A primitive literal are immutable values, meaning they can't be changed to represent other values in the same way that JavaScript's more complex object-based data structures can.

For example, while the value of a variable named `theTruth` can be reassigned a value of `false`, the boolean literal `true` can never represent any value other than `true`, in the same way the number literal `5` can never represent the value of another number.

There are seven primitive data types:

- `Numbers`
- `Strings`
- `Booleans`
- `null`
- `undefined`
- `BigInt`
- `Symbol`

Datatypes and structures

null

The null keyword represents an intentionally defined **absence of value**.

undefined

undefined is a primitive value assigned to **variables** that have just been declared, or to the resulting value of an operation that doesn't return a meaningful value. For example, this can happen when you declare a function in a browser's developer console:

```
function myFunction() {}
```

```
> undefined
```

Variables

Variables

A variable's name is called an identifier. A valid identifier must follow these rules:

- Identifiers can contain Unicode letters, dollar signs (\$), underscore characters (_), digits (0-9), and even some Unicode characters.
- Identifiers can't contain whitespace, because the parser uses whitespace to separate input elements. For example, if you try to call a variable `my Variable` instead of `myVariable`, the parser sees two identifiers, `my` and `Variable`, and throws a syntax error ("unexpected token: identifier").
- Identifiers must start with a letter, underscore (_), or dollar sign (\$). They can't start with digits, to prevent confusion between numbers and identifiers:
 - `let 1a = true;`
 - `> Uncaught SyntaxError: Invalid or unexpected token`

"Reserved words" that are already syntactically meaningful can't be used as identifiers.

Variables

Declaration

There are multiple ways to make JavaScript aware of an identifier, a process called "declaring" a variable. A variable is declared using the let, const, or var keywords.

- Use let instead of var. This is more modern and better supported. Will lead to less issues.

- Use the equal '=' sign to assign or reassign a value to a variable.

- `let myVariable = 5;`
 - `myVariable + myVariable`
 - `> 10`

- **Undefined example**

- `let myVariable;`
 - `myVariable;`
 - `> undefined`
 - `myVariable = 5;`
 - `myVariable + myVariable`
 - `> 10`

Variables - Scoping

- A variable's scope is the part of a script where that variable is available. Outside a variable's scope, it won't be defined—not as an identifier containing an undefined value, but as if it hadn't been declared.
- Depending on the keyword you use to declare a variable and the context in which you define it, you can scope variables to block statements (block scope), individual functions (function scope), or the entire JavaScript application (global scope).
- <https://web.dev/learn/javascript/data-types/variable#scope>

Functions

Functions – Introduction

- A function is a modular, reusable block of statements used to perform a set of related tasks, such as calculating and returning a value based on arguments provided to the function.

```
function myFunction() {  
    console.log( "This is my function." );  
};
```

- A function can be assigned to a variable, passed as an argument to other functions, and returned by other functions.

Functions - Calling

- Referencing a function

```
function myFunction() {  
    console.log( "This is my function." );  
};  
  
myFunction;  
  
> myFunction() {  
    console.log( "This is my function." );  
}
```

- Calling a function

```
function myFunction() {  
    console.log( "My function has been  
executed." );  
};  
  
myFunction();  
  
> "My function has been executed."
```

Looking back at our first js file

```
let http = require('http');  
  
function greetUser(req, res) {  
  res.writeHead(200, {'Content-Type': 'text/html'});  
  res.end('Hello World!');  
}  
  
http.createServer(greetUser).listen(8080);
```

- Is this clearer now?

Helpful Resources

- <https://web.dev/learn>
- <https://www.tutorialrepublic.com/javascript-tutorial/>